

Deep Learning for Visual Computing 2020, Assignment 2

Dominik Schörkhuber
1027470

Thomas Kaufmann
1129115

May 28, 2020

1 Part 1 - Optimizers

In the first part of this assignment we are familiarizing with different optimization algorithms. In Deep Learning we use optimization algorithms, to train the parameters of a neural network. A error (or loss-) function is formulated based on the input and expected output of a network, the optimizer is used to adapt the model parameters such that the loss function is minimized. While Deep Learning problems can typically grow to millions of parameters, we observe in our toy example a 2-dimensional problem space. One can interpret this problem as finding the lowest position on a height map. We visualize the problem space with false color representation, where red areas and blue areas correspond to high values and low values, respectively.

1.1 Stochastic Gradient Descent

We start our investigation with the Stochastic Gradient Descent (SGD) algorithm. Gradient descent refers to stepping along the steepest slope. This idea is the basis for all of the covered optimization techniques. While moving along the gradient is a good heuristic to improve the error value locally, it usually steers towards a local minimum. In the following we describe $L(\theta)$ as the objective function with parameters θ . Gradient descent (like all covered optimization algorithms) works iteratively. In each iteration we compute the velocity v i.e. the change of the parameter vector θ such that $\theta_{new} = \theta_{old} + v$ whereras $v = \beta v - \alpha \nabla L(\theta)$. For a first analysis we assume $\beta = 0$. As such the speed v is composed of the gradient ($\nabla L(\theta)$) and the learning rate α . Higher learning rate means we move further along the gradient in each step, but are therefore less flexible to intermediate changes in the objective function. However, if we set the learning rate too low, the algorithm takes a very long time to converge. Too high learning rate can lead to oscillation as seen in Figure 1b and Figure 1a, respectively.

To increase the speed of convergence we can use the dampening factor β . Successive movements in the same direction build up higher momentum

and allows SGD to take longer steps. The configuration in Figure 2d uses a low dampening factor. Compared to Figure 1b it converges in 74 steps instead of 216 steps, but slightly overshoots at the left turn due to the built up momentum. The effect of too high momentum is shown in Figure 1d. A method to alleviate this is the Nesterov momentum. Instead of computing the gradient at position θ , the gradient is computed at $\theta + v$. This can help because the gradient term always points in the correct direction, while the momentum term may not. This technique severely reduces spiralling around the optimum as shown in Figure 1e.

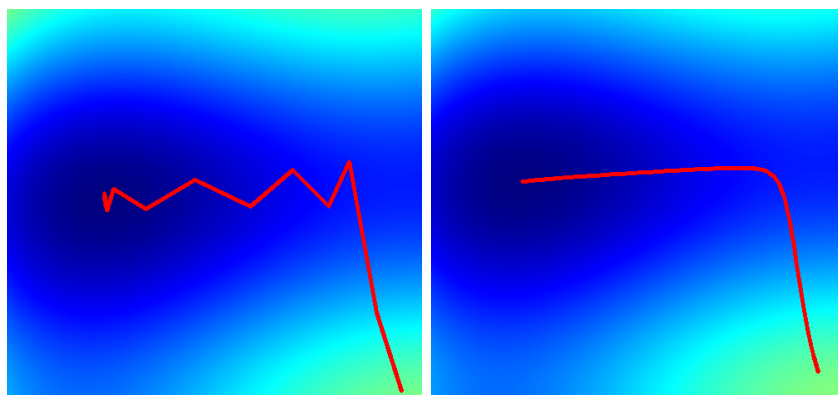
1.2 Comparison of Algorithms

Finally we compare the results of different algorithms on the same optimization problem. Most notably, we expect an optimization algorithm to converge to a minimum, and optimally of course the global minimum. As mentioned in the previous section, the strategy of following gradients is flawed as it steers the algorithm towards a local minimum. Secondly, the algorithm should show fast convergence towards the minimum. Figure 2 shows the results for SGD, RMSProp, Adam and AdamW (with default parameters according to PyTorch, for SGD lr=1) on the Eggholder function. Only AdamW (due to the by default high weight decay) is able to move past the local optimum to finally reach a better overall result. For this function it seems that only a different starting point would lead to better results for most algorithms. Figure 2 also nicely shows the behaviour near saddle points. At a saddle point the gradients become very low which considerably impacts convergence speed. To visualize this see the error over steps plotted in Figure 2e

2 Part 2 & 3 - Cats & Dogs

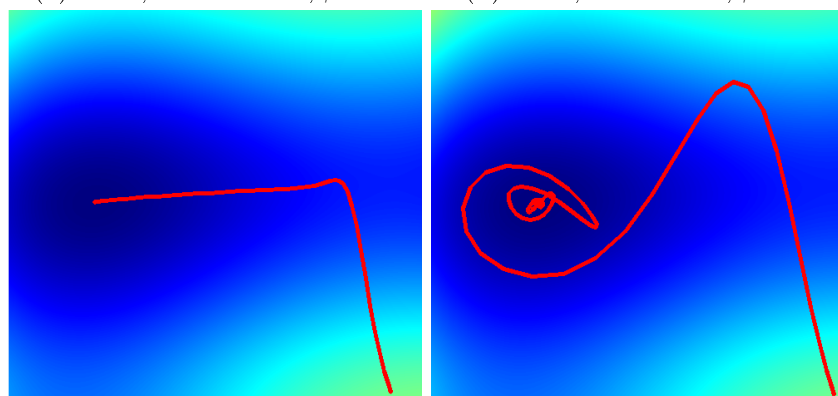
2.1 Architectures

We started part two with a rather simple architecture consisting of two convolutional layers with max pooling as well as three linear layers with *relu* activation functions. Although for this very simple model the training error converged towards 0 rather soon after about 60 epochs, the accuracy remained rather moderate at approximately 65%. We then turned our focus



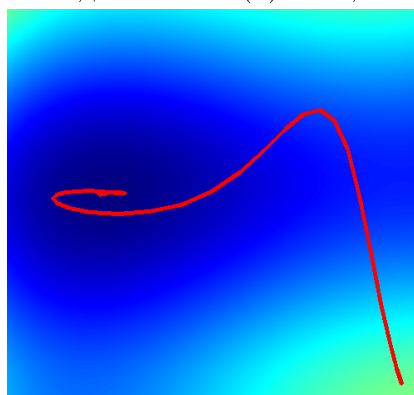
(a) SGD, $lr = 100000$, $\beta = 0$

(b) SGD, $lr = 10000$, $\beta = 0$



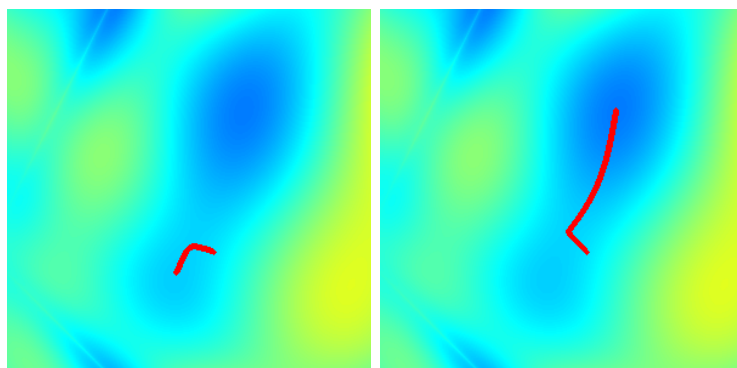
(c) SGD, $lr = 10000$, $\beta = 0.5$

(d) SGD, $lr = 10000$, $\beta = 0.9$



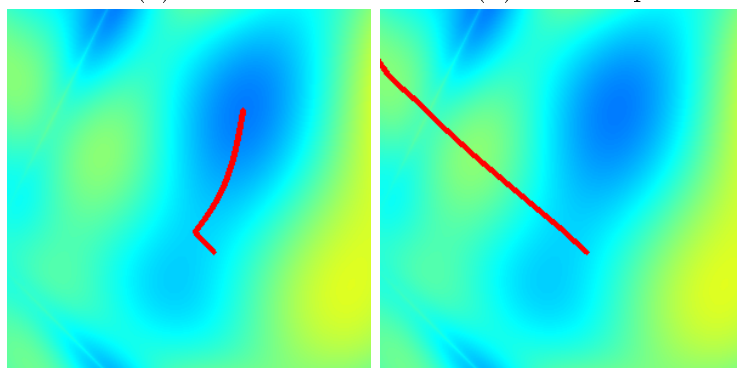
(e) SGD, $lr = 10000$, $\beta = 0.9$,
with Nesterov momentum

Figure 1: Shown are different configurations for SGD to emphasize the effect of different parameters.



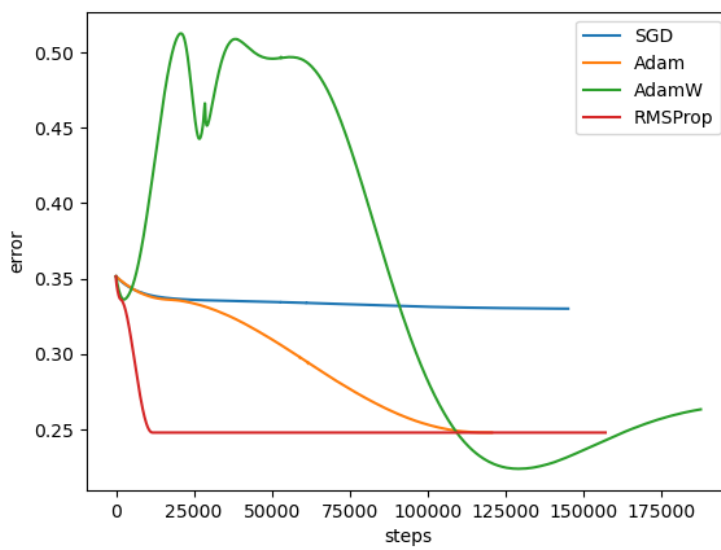
(a) SGD

(b) RMSProp



(c) Adam

(d) AdamW



(e) Loss over steps

4
Figure 2: Comparison of different optimizers

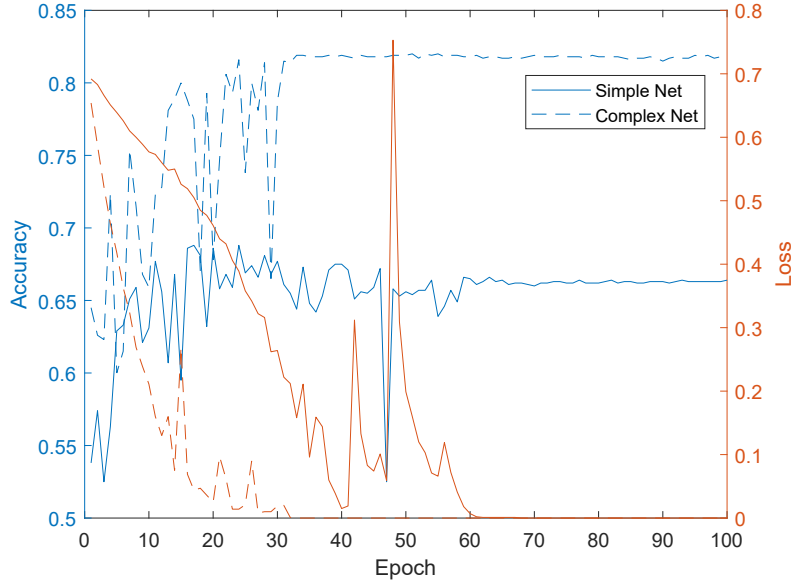


Figure 3: Comparison of a simple and a complex CNN

on a significantly larger and deeper network¹, again employing convolutional and max pooling layers, followed by a significantly larger linear layer with *relu*. Figure 3 shows that already the deeper architecture without any enhancements introduced in Part 3 lead to a significant improvement of the accuracy above 80%, while the training loss still converges towards 0, indicating that an over-fitted network.

2.2 Augmentation

Data augmentation is a technique commonly applied in machine learning to artificially diversify the training set by introducing randomized perturbations on the samples with operators natural to the a sample's structure, like brightness modifications, crops for imaging data or even shifts or stretches for audio. The randomized nature of augmentation then prevents the net from *seeing* the very same sample several times in the training process, aiming to avoid overfitting fostering a more generalizing behavior of the net.

Figure 7 illustrates the applied augmentations. Starting with the original

¹<https://zhenye-na.github.io/2018/09/28/pytorch-cnn-cifar10.html>

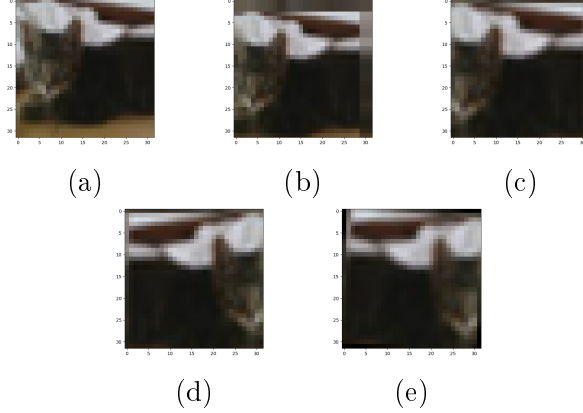


Figure 4: Augmentation pipeline

image in Figure 4a subject to a per channel average normalization, we apply a random crop of a 32×32 square with mean padding of 4 in second stage shown in Figure 4b, followed by center crop of 28×28 with a probability of 0.05 with inter cubic upscaling to our networks resolution of 32×32 . Step four illustrated in Figure 4d performs a left/right mirroring, followed by a randomized rotation with an angle of $[-5^\circ, +5^\circ]$.

2.3 Regularization

Regularization in machine learning are techniques aiming to improve the generalizing behavior of an algorithm by reducing its generalization error, while not necessarily the training error. In the literature a variety of such techniques can be found, affecting the an algorithms parameters, introducing additional constraints or introducing extra terms in the objective function. In weight decay regularization, the loss function is extended by the so-called regularization term, being the weighted Euclidean Norm of the weight matrices. As regularization weight δ we use the values $\delta_1 = 5e^{-4}$ being moderately sized in the usual spectrum, while $\delta_2 = 0.01$ is already representative of the upper end of the spectrum, tending to dominate the loss function. Alternatively, we also make use of dropout which another commonly used regularization mechanism, where during training certain units output 0 with a probability of p inducing similarities to ensemble learning principles, where multiple independent models are combined. In this project, we use dropout before the dense linear layers with a probability of 0.1 as well as in our more

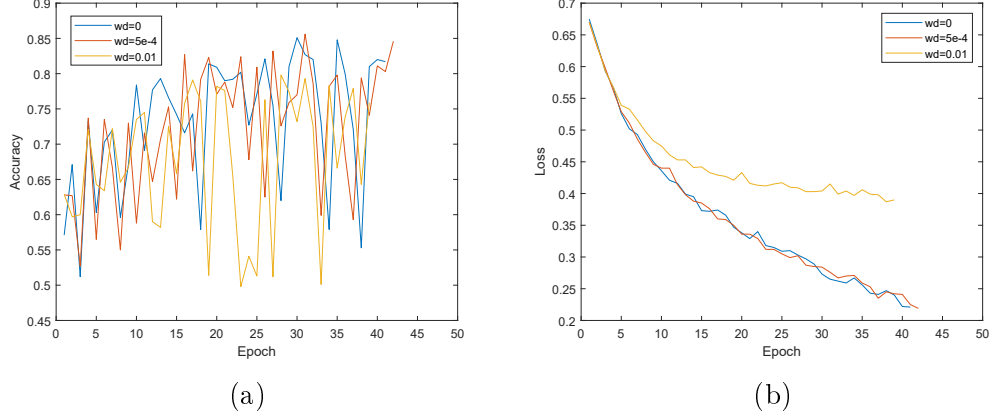


Figure 5: Comparison weight decay

complex network in the convolutional layer with a probability of 0.05.

Early stopping aims to avoid overfitting by prematurely terminating the training phase based on metrics obtained from the validation set in order to approximate a point where the model is well enough trained before overfitting sets in. In this work, we used an upper bound of 100 epochs and experimented with patience levels of 5 and 10 for early stopping.

2.4 Experimental Results

In experimental analysis conducted in part 3, we decided to focus on the more complex architecture and performed experiments with differently weighted decays, different levels of data augmentation and enabled dropout. Generally, it can be observed that our early stopping approach sets in already

Figure 7 compares accuracy and training loss of our complex net with full data augmentation and enabled dropout for weights $\delta_0 = 0$, $\delta_1 = 5e^{-4}$ and $\delta_2 = 0.01$. It can be observed that a dominating weight decay $\delta_2 = 0.01$ can negatively affect the accuracy, while being able to avoid premature convergence of training loss. Furthermore, in our setup a moderate $\delta_1 = 5e^{-4}$ behaves rather similarly both with respect to accuracy as well as training loss as $\delta_0 = 0$, which may be an indicator that other measures to avoid overfitting, as dropout or augmentation suffice in this case, thus no significant improvement can be observed.

Figure 7 compares accuracy in the validation set as well as training loss

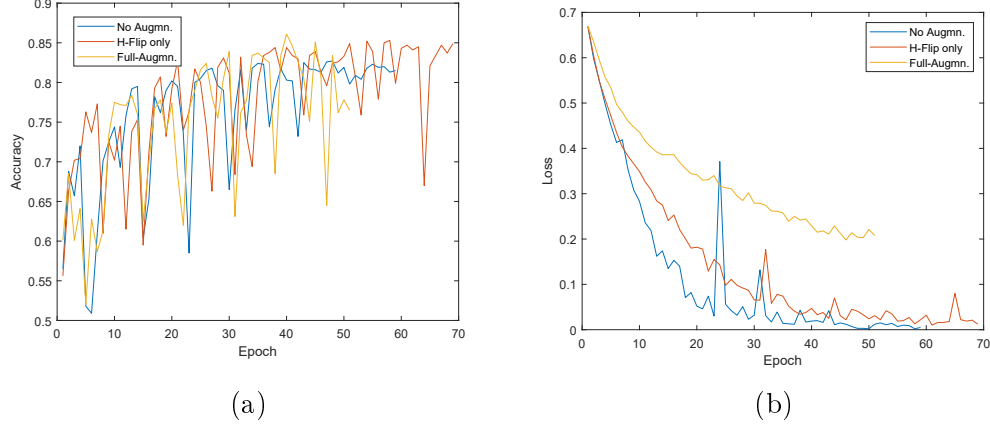


Figure 6: Comparison augmentation strategies

for three different augmentation settings. First it can clearly be observed that training loss converges faster compared to the augmented setups, with substantially inferior accuracy, clearly illustrating the positive effect of our data augmentation strategies. Even more our advanced augmentation approaches show further improvement, both in avoidance of premature convergence as well as peak accuracy.

Finally, we compared two variations of our complex network with dropout enabled and disabled. In this case, it seems that our early stopping patience seems to be too short and prematurely stops the optimization. Without dropout the optimization steadily improves for another 40 epochs obtaining in the end a superior accuracy. However, since in this setup both augmentation as well as weighted decay was enabled, these means against overfitting may be sufficient.

2.5 Transfer Learning

Transfer learning describes the technique of transferring the parameters of one network to another. This can be useful to apply networks, or parts of networks, to a different domain of data or an entirely different problem. PyTorch provides a range of pre-trained networks for us. These networks have been trained for image classification on the ImageNet dataset. We can reuse most of the network and only swap out the final layer for a fully connected layer to output our binary classification result. For our experiments we apply

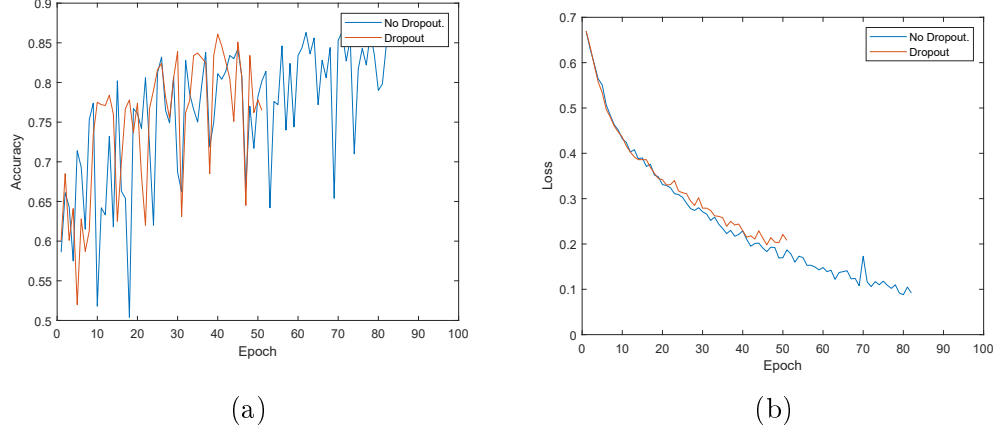
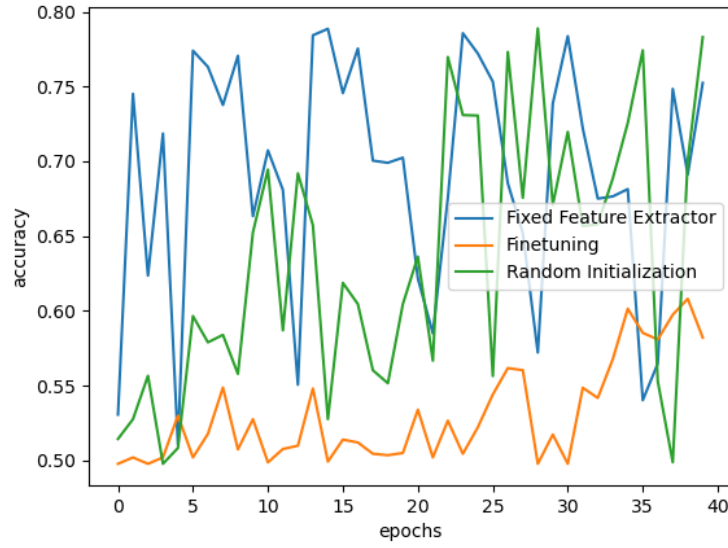
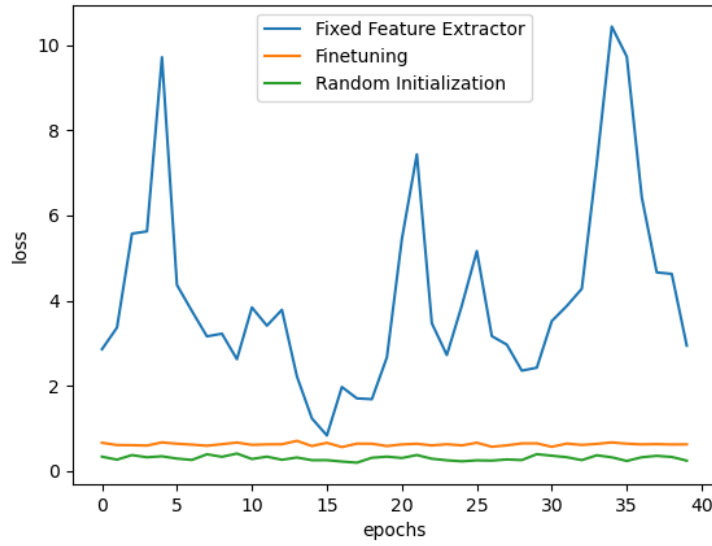


Figure 7: Comparison dropout approaches

ResNet18 and freeze all layers, except for the mentioned output layer and train the network for 100 epochs. We compare that technique to finetuning, which also utilizes the transferred weights but retrain all layers during training. Finally, we compare both of those configurations to a model trained with random initialization. The resulting loss and accuracy curves can be observed in Figure ?? . Due to the pre-trained weights in the backbone of our network, we reach high accuracy faster compared to random parameter initialization. However, random initialization still produced the best performing model after around 25 epochs.



(a) Accuracy



(b) Loss

Figure 8: Loss and accuracy for random weight initialization, fine tuning and frozen backbone layers